Testing and debugging functions

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

Before you start

For capacity 9 and used 2, what should remaining return?

Plan time to read, investigate and explain. Keep predictions separate from observations.

Your mission

> Start with the target: Design revealing tests for a function. Try the recall before opening notes.

Your learning target

I can design revealing tests for a function.

I can compare expected and actual results.

I can repair a defect and rerun earlier cases.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Design revealing tests for a function

Compare expected and actual results

Repair a defect and rerun earlier cases

Retrieve what you already know

> For capacity 9 and used 2, what should remaining return?

My starting explanation

This week's end goal

Trace testing and debugging functions and explain the changed program before running it.

Notice the evidence

> Investigate the supplied Testing and debugging functions case. Separate observations, given facts and assumptions before choosing an explanation.

A function can pass one example by coincidence. Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases.

We will test a subtraction function under the contract 0 <= used <= capacity.

> **List the given facts and one fact that is not supplied.**

STUDENT LESSON · 3 OF 18

See how it works

> Follow the mechanism for Testing and debugging functions. Connect each labelled input, deciding step and result.

```
def remaining(capacity, used):
    return capacity - used

cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual == expected)
```

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Ordinary case	8 and 3	Expect 5
Full case	8 and 8	Expect 0
Unused case	8 and 0	Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result.

The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with **Assertion**Error.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

STUDENT LESSON · 4 OF 18

Compare the cases

> Compare the supplied cases for Testing and debugging functions. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

A faulty function always returns 4. Which of your three cases pass and fail?

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change one thing

Change: Remove the test at 3 and keep only 2 and 4.

Predicted result

Prediction: The buggy <= function passes those remaining tests.

Why the result follows

Reason: Both procedures agree away from the equality **boundary**.

Words that unlock the idea

> Use the vocabulary of Testing and debugging functions to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Test case	Inputs and an expected result
Boundary	An edge of a permitted range
Regression	A previously working case that fails after a change
Assertion	A check that raises an error when its condition is false

> **Use two terms to explain a step in this week's example.**

STUDENT LESSON · 6 OF 18

Follow a worked example

> Follow the worked trace for Testing and debugging functions, predict a step, then compare with its explanation.

Would replacing the body with return 5 pass every worked case? Identify the first revealing test.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Testing and debugging functions

Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

The facts we will use

Supplied facts

Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False). Candidate bug: return x<=3.

AI ACADEMY / PYTHON-BRIDGE/12

Testing and debugging functions

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Record contract

Strict less-than excludes the value 3.



2. Run boundary cases

The intended outputs are True, False, False for 2,3,4.



3. Locate the bug

Using <= passes 2 but fails the test at 3 before the later test.

CHANGE ONE THING

Remove the test at 3 and keep only 2 and 4.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Record contract	Strict less-than excludes the value 3.
2. Run boundary cases	The intended outputs are True, False, False for 2,3,4.
3. Locate the bug	Using <= passes 2 but fails the test at 3 before the later test.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

What this example does not establish

Keep the stated limit

Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

Keep the stated limit

Read the labelled picture: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

Keep the stated limit

Change one thing: Remove the test at 3 and keep only 2 and 4. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Testing and debugging functions claim and explain a repair using the supplied evidence.

A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases.

Never change an expected result merely to match faulty **output**.

Locate the earliest unsupported step

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

State one repair and the checks you would rerun.

STUDENT LESSON · 8 OF 18

Practise together

> Practise Testing and debugging functions with support, then explain the deciding step in your own words.

Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

Would replacing the body with return 5 pass every worked case? Identify the first revealing test.

Explain your prediction to a partner before comparing results.

Practise with less help: Find a revealing case

Use workbook W2 for the connected example “Testing and debugging functions”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

> Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

Run the investigation

> Investigate Testing and debugging functions. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

> A faulty function always returns 4. Which of your three cases pass and fail?

```
def remaining(capacity, used):
    return capacity - used

cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual == expected)
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Testing and debugging functions. Explain why each step is justified.

Explain → apply → challenge

Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

> All three stock tests pass. Does that prove correct handling of negative counts?

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled testing and debugging functions **evidence** sheet to Python readiness portfolio. Show how you can design revealing tests for a function.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling. A test outside the contract needs a separately specified expected behavior.

All three stock tests pass. Does that prove correct handling of negative counts?

All three stock tests pass. Does that prove correct handling of negative counts?

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

Revisit the complete Testing and debugging functions case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Ordinary case	8 and 3	Expect 5
Full case	8 and 8	Expect 0
Unused case	8 and 0	Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

STUDENT LESSON · 14 OF 18

Try a new case independently

Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Use boundary tests to reveal a function defect

Predict the **output**. If the function mistakenly uses x<=10, identify the first failing **assertion** and whether the final print happens. State the repair and which earlier tests must be rerun.

Do these cases prove all **input** types are handled?

```
def below_limit(x):
    return x < 10
assert below_limit(9) is True
assert below_limit(10) is False
assert below_limit(11) is False
print("three checks passed")
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

STUDENT LESSON · 15 OF 18

Your lesson route

> Use this map to navigate Testing and debugging functions, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

By the end, I can	How I will show it
Design revealing tests for a function	Explain the deciding step, show a trace or test, and state one limit.
Compare expected and actual results	Explain the deciding step, show a trace or test, and state one limit.
Repair a defect and rerun earlier cases	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Testing and debugging functions without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Design revealing tests for a function

> **My deciding step, evidence and one remaining question**

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? Using <= passes 2 but fails the test at 3 before the later test.

1. Locate the bug

2. Record contract

3. Run **boundary** cases

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Locate the bug Using <= passes 2 but fails the test at 3 before the later test. A revealing test distinguishes a correct implementation from a plausible mistake.

Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Remove the test at 3 and keep only 2 and 4. Which conclusion is supported by the supplied facts?

1. The buggy <= function passes those remaining tests.

2. This one example guarantees correct results for every future **input**.

3. We can decide the result without examining the changed **input** or the procedure.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: The buggy <= function passes

> those remaining tests. Both procedures agree away from the equality **boundary**.

Repair a missing building block

Review Reading structured data (python-bridge/11). Goal: **Process** a list of records consistently. Short practice: Circle the included rows, calculate the sum and mean, and state coverage.

Repeat after adding the fourth row with count four.. Return to this week and explain what you changed.

Review Return values and local state (python-bridge/9). Goal: Use a returned value in a later calculation. Short practice: Trace the two returned values and explain why first stays seven.

What happens if the caller tries to print result directly?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

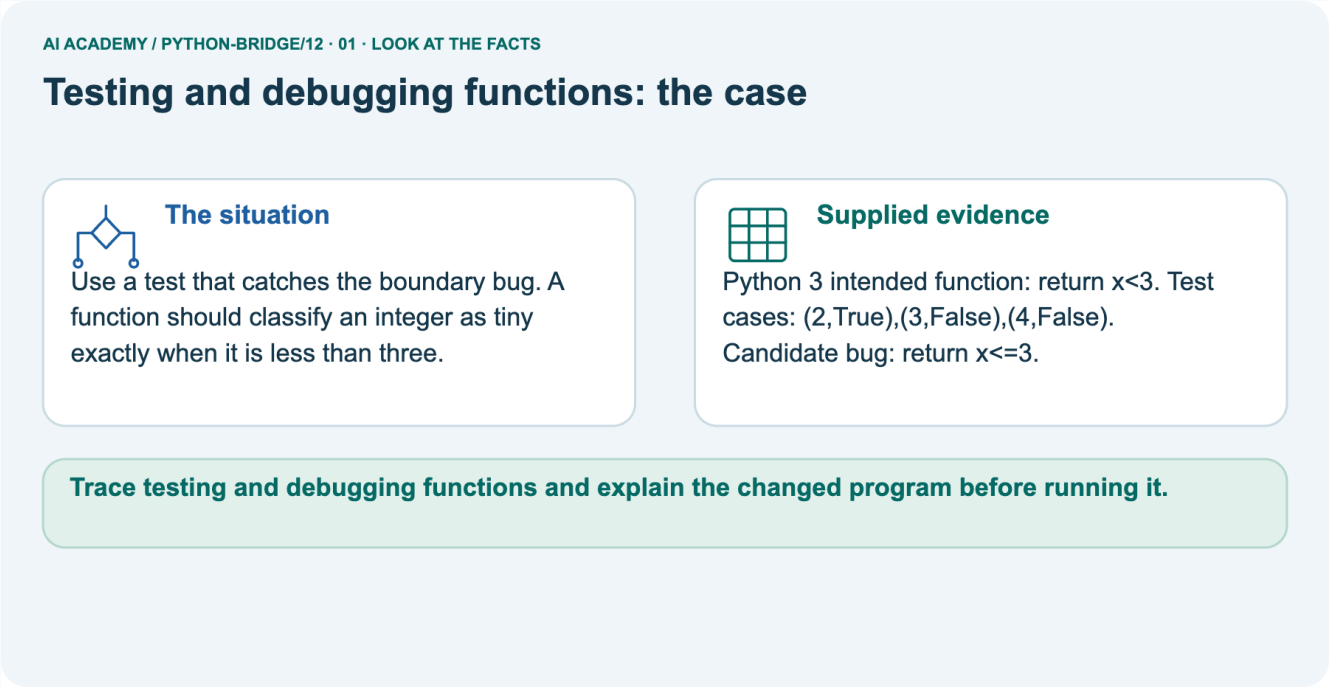
Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

Visual concept journey

> Explain each Testing and debugging functions graphic using its labels, deciding evidence and relevant limitation.

01. Testing and debugging functions: the case



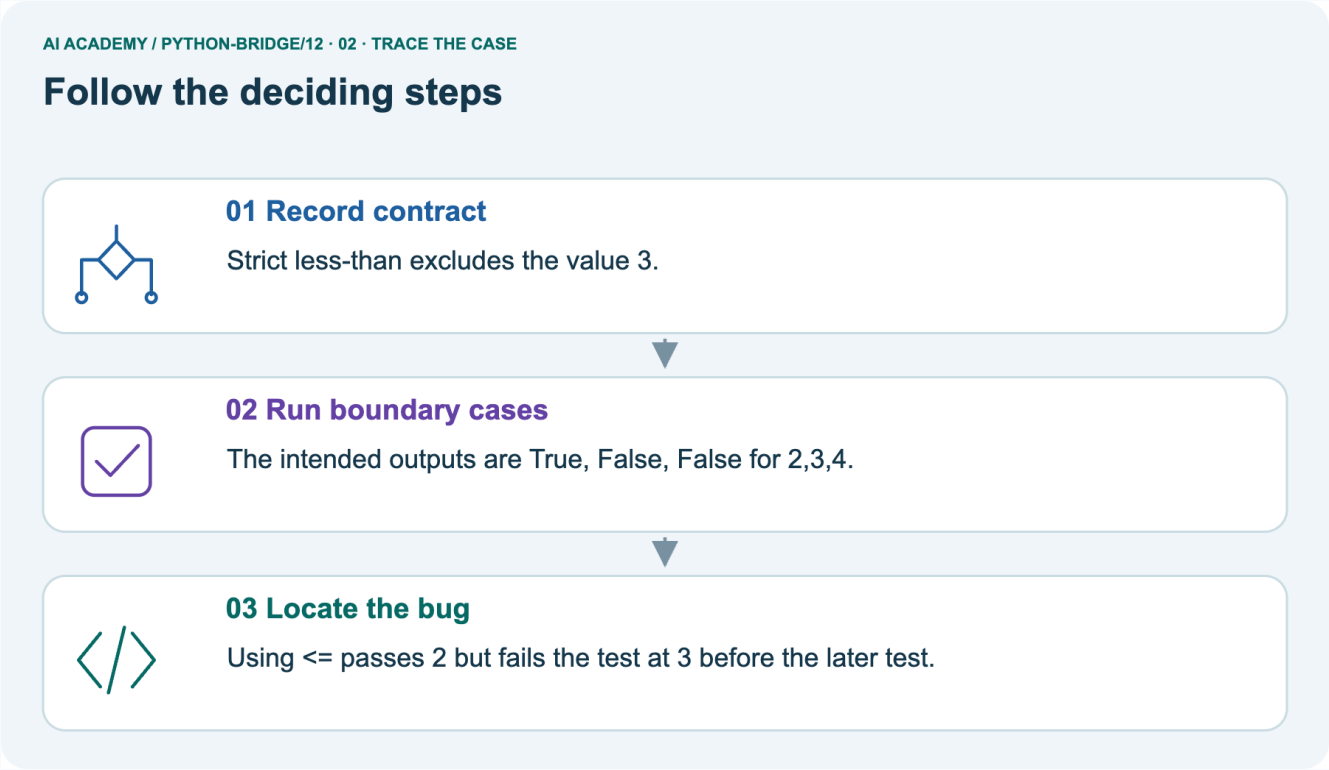
Follow the labelled picture with the matching explanation. Open the vector diagram

Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three. Supplied facts: Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False).

Candidate bug: return x<=3.

> Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

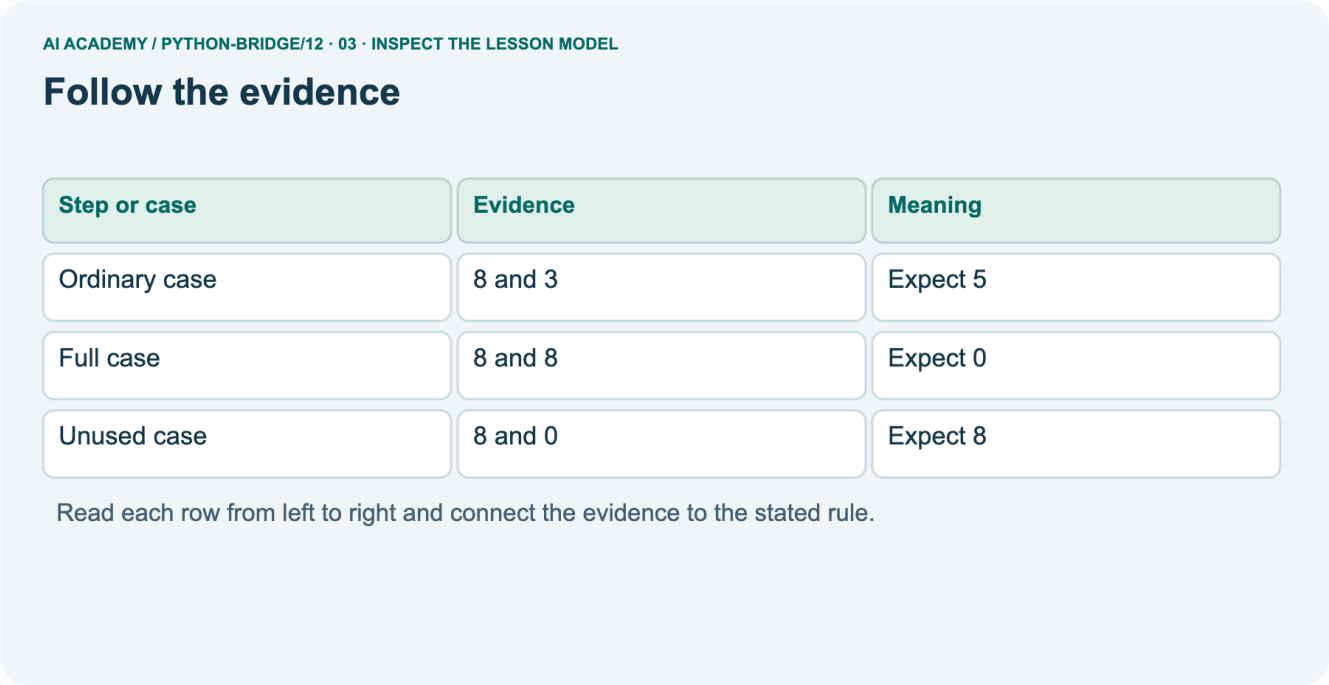


Follow the labelled picture with the matching explanation. Open the vector diagram

Record contract: Strict less-than excludes the value 3. → Run **boundary** cases: The intended **output**s are True, False, False for 2,3,4. → Locate the bug: Using <= passes 2 but fails the test at 3 before the later test.

> Point to each numbered stage and explain which recorded **input** it uses.

03. Follow the evidence

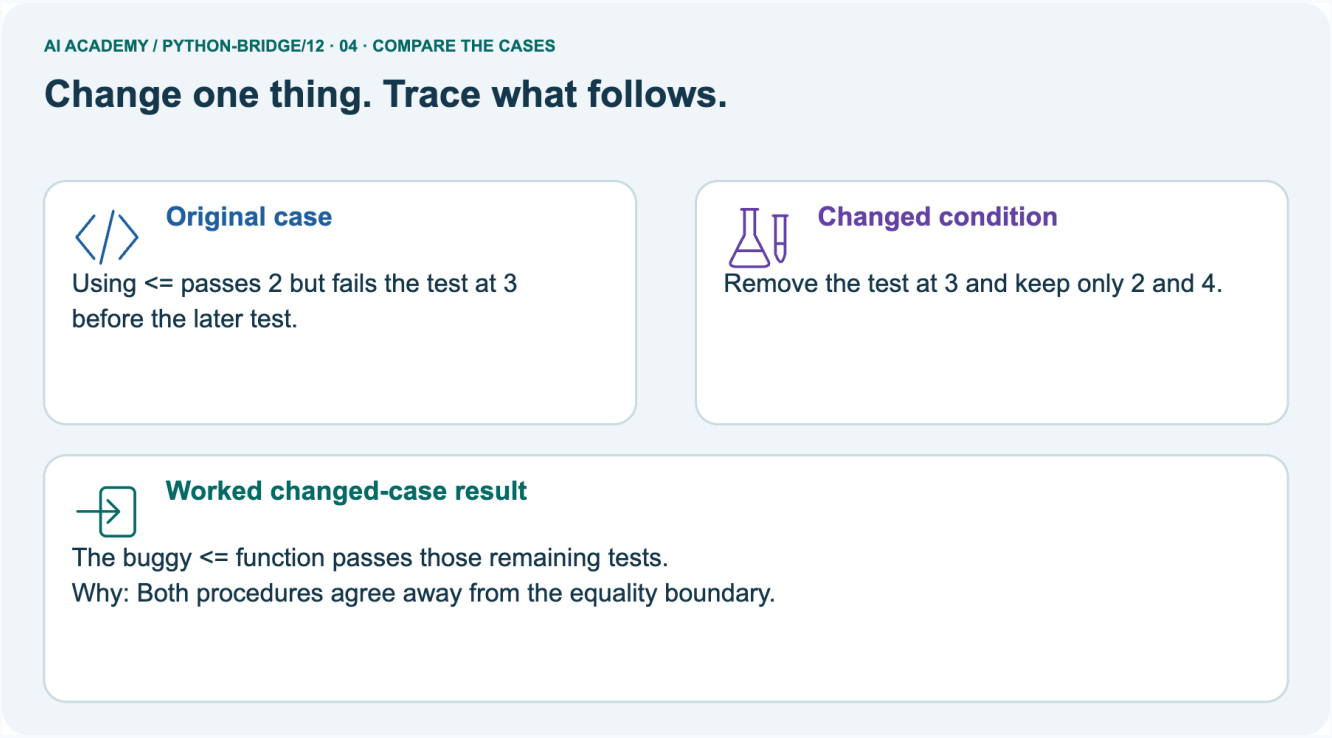


Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Ordinary case | 8 and 3 | Expect 5. Full case | 8 and 8 | Expect 0. Unused case | 8 and 0 | Expect 8

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



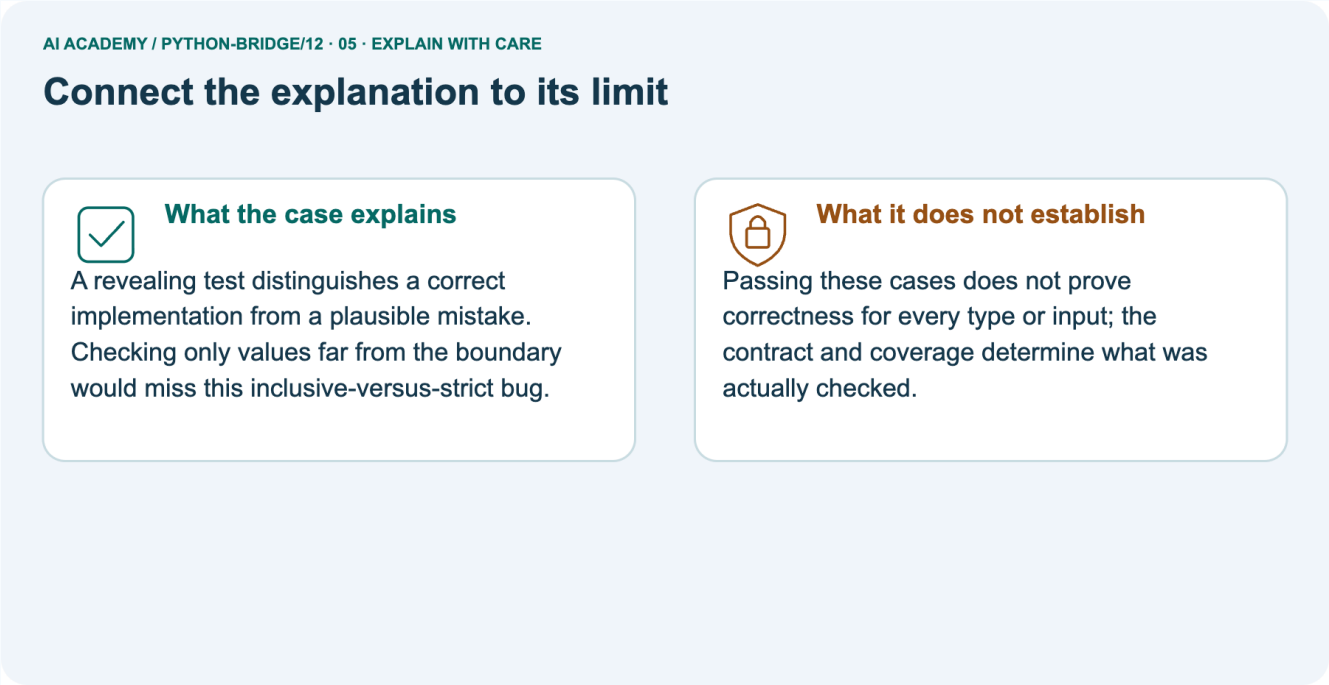
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Using <= passes 2 but fails the test at 3 before the later test. Changed condition: Remove the test at 3 and keep only 2 and 4. Expected result: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



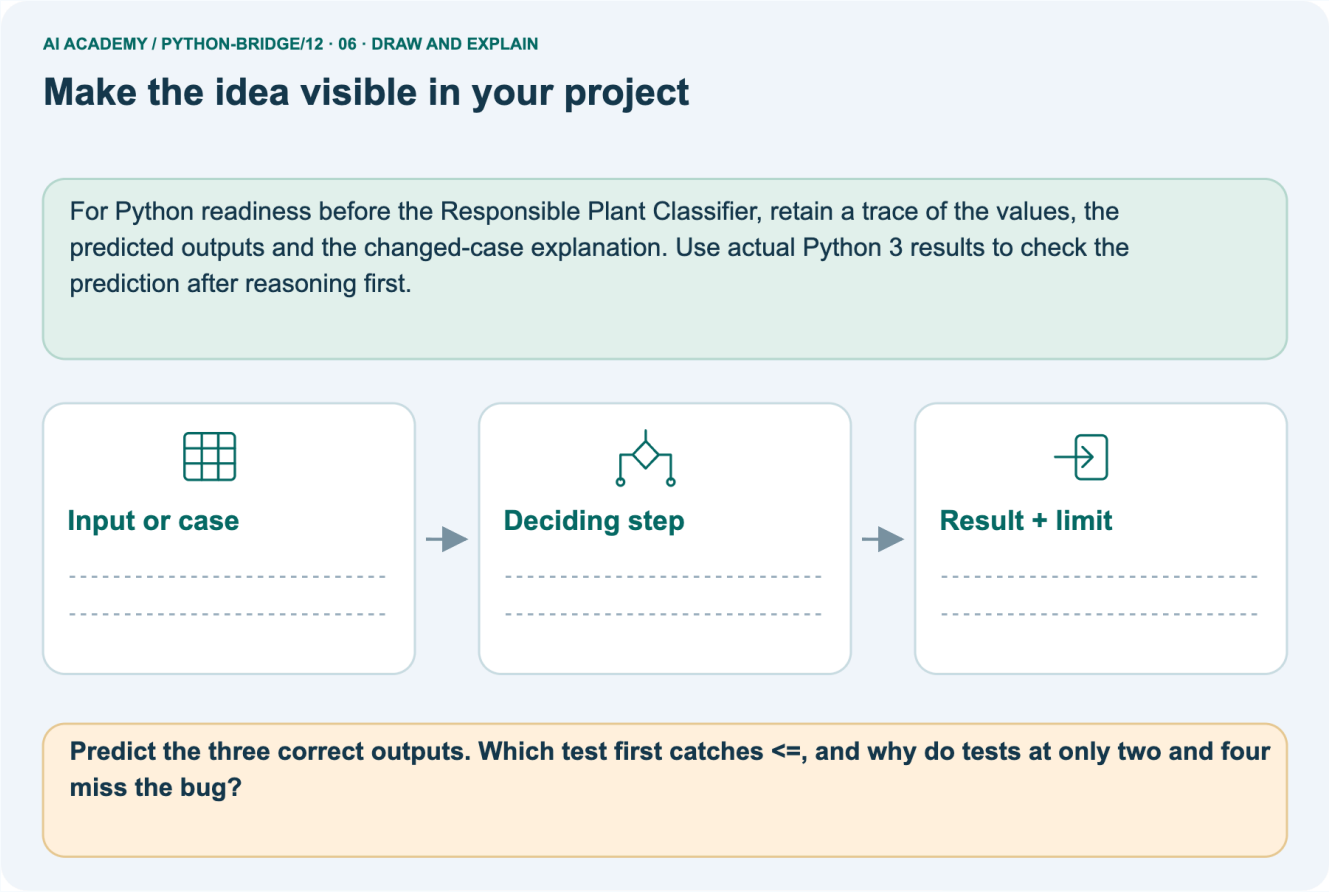
Follow the labelled picture with the matching explanation. Open the vector diagram

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

Limit: Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.